Maths for programming

1) count didgits in n

```
int n = 777;
int count = 0;

while (n)
{
   count++;
   n /= 10;
}
cout << "total didgits in n are : " << count;
```

method 2)

```
int n=889;
   int cnt = (int)(log10(n) + 1);
   cout << cnt;
}
```

2) Reverse a number :

```
int n;
   cin >> n;
   int revNum = 0;
   while (n)
   {
      int lsdigit = n % 10;
      revNum = revNum * 10 + lsdigit;
      n /= 10;
   }
   cout << "reverse Number is : " << revNum;
```

3) armstong number

```
int arms = 0;
   int orignal = n;
   while (n)
   {
      int j = n % 10;
      arms = arms + (j * j * j);
      n /= 10;
   }
   if (orignal == arms)
      cout << "Its Armstrong number";
   else
      cout << "Not Armstrong number";
```

4) finding factors of n

```
vector<int> v;
for (int i = 1; i < sqrt(n); i++)
{
    if (n % i == 0)
    {
        v.push_back(i);
        if ((n / i) != i)                    //x*y means i * (n/i)
        {
            v.push_back(n / i);
        }
    }
}
sort(v.begin(), v.end());
for (auto it : v)
    cout << it << " ";
```

4) prime number

```
int n = 15;
    int cnt = 0;
    for (int i = 1; i * i <= n; i++)
    {
        if (n % i == 0)
        {
            cnt++;
            if ((n / i) != i)
                cnt++;
        }
    }
    if (cnt == 2)
        cout << "Prime Number";
    else
        cout << "Not Prime Number";
```

5) GCD

Naive approach: run for loop from 1 to min(n,m) and check if i is div by both n,m
return max value of i that divides n,m

optimal solution:

Euclidean Algorithm

gcd(a,b) = gcd(a%b,b) ; a>b

```
int a = 150, b = 120;
   while (a > 0 && b > 0)
   {
      if (a > b)
         a = a % b;
      else
         b = b % a;
   }
   if (a == 0)
      cout << b;
   else
      cout << a;
```

6) Sieve of Eratosthenes: to find prime numbers till given number

or precomposition. * maintain a vaector of size n+1
(keep vector of index * mark all index as 1
if prime index has value
equal 1) * if index is not prime assign it 0

```
// There are no primes strictly less than 2
        if (n <= 2)
            return 0;
        // Maintain a vector of size n+1, mark all index as 1
        vector<int> arr(n + 1, 1);
        // 0 and 1 are not prime, so assign them 0
        arr[0] = 0;
        arr[1] = 0;
        // Traverse from 2 up to the square root of n
        for (int i = 2; i * i < n; i++) {
            // If the index is marked as 1, it is a prime
            if (arr[i] == 1) {
                // Mark all multiples of i as 0 (not prime)
                for (int k = i * i; k < n; k += i) {
                    arr[k] = 0;
                }
            }
        }
        // Count all the indices that are still marked as 1
        int count = 0;
        for (int i = 2; i < n; i++) {
            if (arr[i] == 1) {
                count++;
            }
        }

        return count;
```